МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)
Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Реализация численных алгоритмов

Студент
Гр. БИН-26-03 _____ М. П. Гришин
Студент
Гр. БИН-26-03 _____ Т. Н. Бобров
Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Ход работы

1.1 Задание 1. Наибольший общий делитель

Пользователь вводит два целых числа a и b на отдельных строках. Если хотя бы одно число отрицательное или оба числа равны нулю, выводится “Ошибка”. Для остальных значений необходимо найти наибольший общий делитель (НОД) алгоритмом Евклида.

Блок-схема алгоритма программы для задания 1 представлена на рисунке 1.

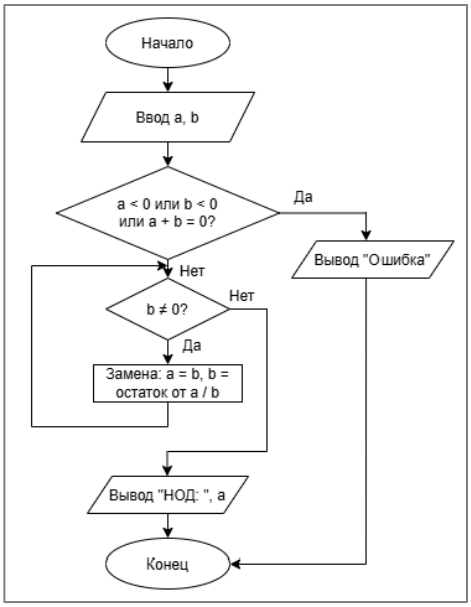


Рисунок 1 - Блок-схема алогоритма для задания 1

Текст программы для задания 1 представлен на рисунке 2.

```
a = int(input())
b = int(input())

if a < 0 or b < 0 or a + b == 0:
    print("Ошибка")
else:
    while b != 0:
        a, b = b, a % b
    print("НОД:", a)
```

Рисунок 2 – Текст программы для задания 1

Входные данные: 84 30

Результат: 6

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.2 Задание 2. Проверка числа на простоту

Пользователь вводит целое число n. Необходимо вывести “Простое” или “Не простое”. Простое число - целое число больше еденицы, которое имеет ровно два положительных делителя. Все значение меньше 2 дают результат “Не простое”. Для проверки перебираются возможные делители начиная с 2. Проверка завершается при нахождении делителя или когда квадрат проверяемого делителя превысит n.

Блок-схема алгоритма программы для задания 2 представлена на рисунке 3.

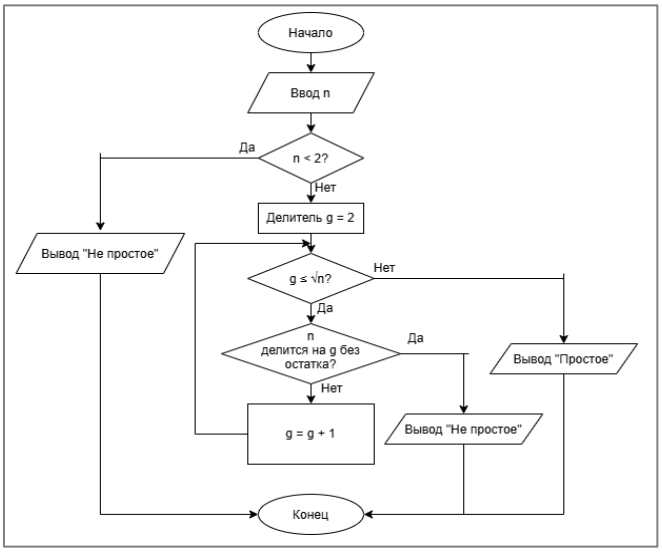


Рисунок 3- Блок-схема алгоритма для задания 2

Текст программы для задания 2 представлен на рисунке 4.

```
n = int(input())
if n < 2:
    print("Не простое")
else:
    for g in range(2, int(n**0.5) + 1):
        if n % g == 0:
            print("Не простое")
            break
    else:
        print("Простое")
```

Рисунок 4 – Текст программы для задания

Входные данные: 49

Результат: Не простое

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.3 Задание 3. Разложение на простые множители

Пользователь вводит целое число n. Если n < 2, выводится “Ошибка”. Необходимо разложить допустимое число на простые множители и вывести их в порядке неубывания на одной строке через один пробел, без пробела в конце. Используется последовательное деление: перебирать делители от 2, при делении без остатка уменьшать оставшееся число и повторно проверять тот же делитель.

Блок-схема алгоритма программы для задания 3 представлена на рисунке 5.

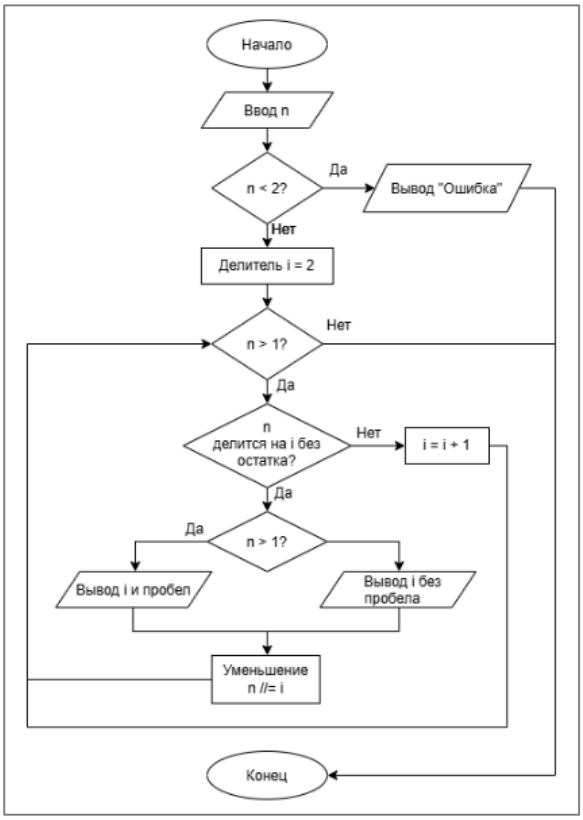


Рисунок 5 - Блок-схема алгоритма для задания 3

Текст программы для задания 3 представлен на рисунке 6.

```
n = int(input())
if n < 2:
    print("Ошибка")
else:
    i = 2
    while n > 1:
        if n % i == 0:
            print(i, end=" " if n > i else "")
            n //= i
        else:
            i += 1
```

Рисунок 6 – Текст программы для задания 3

Входные данные: 60

Результат: 2 2 3 5

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.4 Задание 4. Цифры числа

Пользователь вводит целое число n. Если число отрицательное, выводится “Ошибка”. Для допустимого значения необходимо определить количество цифр и их сумму, последовательно выделяя последнюю цифру остатком от деления на 10 и удаляя ее целочисленным делением на 10. У числа 0 одна цифра и сумма цифр равна 0.

Блок-схема алгоритма программы для задания 4 представлена на рисунке 7.

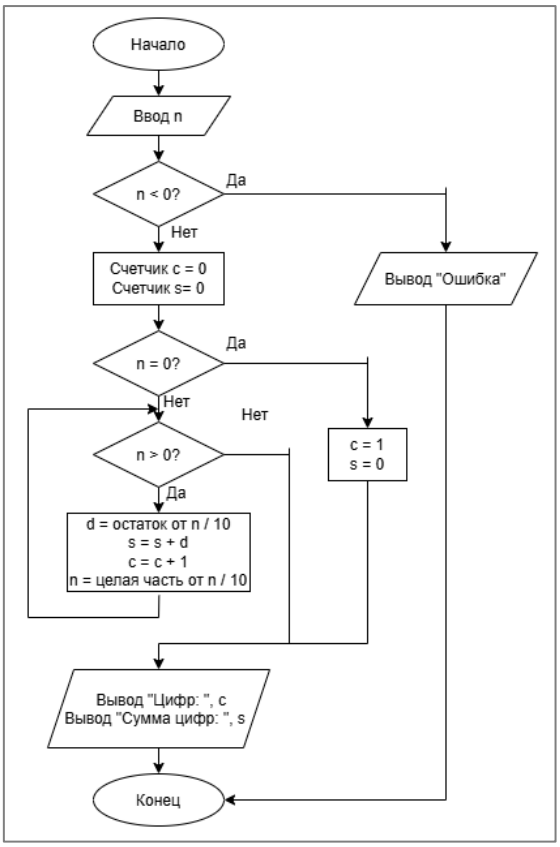


Рисунок 7 - Блок-схема алгоритма для задания 4

Текст программы для задания 4 представлен на рисунке 8.

```
n = int(input())

if n < 0:
    print('Ошибка')
else:
    c = 0
    s = 0
    if n == 0:
        c = 1
        s = 0
    else:
        while n > 0:
            d = n % 10
            s += d
            c += 1
            n = n // 10

    print(f'Цифр: {c}')
    print(f'Сумма цифр: {s}')
```

Рисунок 8 – Текст программы для задания 4

Результат работы программы при вводе 4050 представлен на рисунке 9.

```
Цифр: 4
Сумма цифр: 9
```

Рисунок 9 - Результат работы программы для задания 4

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.5 Задание 5. Таблица расстояний

Пользователь вводил целое число n от 1 до 10 включительно. При выходе за этот диапозон выводится “Ошибка”. На прямой расположены точки с целочисленными координатами от 0 до n – 1. Необходимо построить таблицу расстояний между каждой парой точек. В строке с номером i и столбце с номером j находится расстояние между точками i и j,

равное большей координате минус меньшая. Строки и столбцы выводятся в порядке возрастания координат, без заголовков, элементры строки разделяются одним пробелом.

Блок-схема алгоритма программы для задания 5 представлена на рисунке 10.

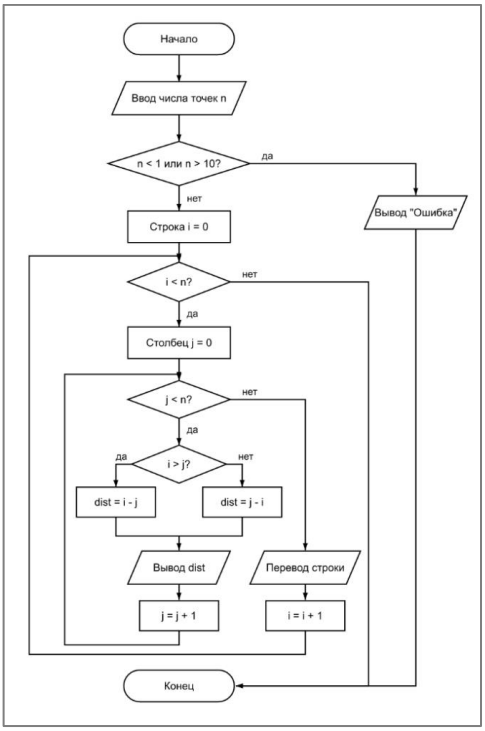


Рисунок 10 - Блок-схема алгоритма для задания 5

Текст программы для задания 5 представлен на рисунке 11.

```
n = int(input())

if n < 1 or n > 10:
    print("Ошибка")
else:
    for i in range(n):
        row = ""
        for j in range(n):
            if i > j:
                d = i - j
            else:
                d = j - i
            if j > 0:
                row += " "
            row += str(d)
        print(row)
```

Рисунок 11 – Текст программы для задания 5

Результат работы программы при вводе 4 представлен на рисунке 12.



Рисунок 12 - Результат работы программы для задания 5

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.